ICPC Code Template

喵喵喵幼儿园

8 CATEGORIES **22** TEMPLATES **876** SOURCE LINES

Quick Index

1 Data Structures

1.1 01 树状数组

FILE data-structures/fwt_bitset.cpp **LINES** 41

USE 下标从 1 开始，激活位置并查询区间内已激活位置数 **COST** Add/query O(log(N/64)), in-block query O(1)

AUTHOR ppip

```
struct fwt {
    static constexpr int B{(N>>6)+5};
    int t[B],n;
    unsigned long long b[B];
    void clear() {
        n=(::n>>6)+1;
        fill_n(t,n+1,0);
        fill_n(b,n+1,0);
    }
    int px(int x) {
        int y{x&63},z{x>>6};
        return __builtin_popcountll(b[z]&(~0ull>>(63-y)));
    }
    int qry(int l,int r) {
        --l;
        int ans{px(r)-px(l)};
        l>>=6;r>>=6;
        while (r>l) ans+=t[r],r&=r-1;
        while (l>r) ans-=t[l],l&=l-1;
        return ans;
    }
    void add(int x) {
        unsigned long long w{1ull<<(x&63)};
        if (b[x>>6]&w) return;
        b[x>>6]|=w;
        x=(x>>6)+1;
        while (x<=n) {
            ++t[x];
            x+=x&-x;
        }
    }
    int qry(int x) {
        int ans{px(x)};
        x>>=6;
        while (x) {
            ans+=t[x];
            x&=x-1;
        }
        return ans;
    }
} t;
```

1.2 O(1) 查询树状数组

FILE data-structures/fwt_suffix.cpp **LINES** 16

USE 下标从 1 开始，单点加、查询后缀和；适合查询远多于修改 **COST** Query O(1), add O(128+N/4096)

AUTHOR ppip

```
struct FWT {
    int s1[N+5],s2[(N>>6)+5],s3[(N>>12)+5];
    void init() {
        memset(s1,0,sizeof(s1));
        memset(s2,0,sizeof(s2));
        memset(s3,0,sizeof(s3));
    }
    void add(int x,int y) {
        for (int i{(x>>6)<<6};i<=x;++i) s1[i]+=y;
        for (int i{(x>>12)<<6};i<(x>>6);++i) s2[i]+=y;
        for (int i{0};i<(x>>12);++i) s3[i]+=y;
    }
    int qry(int x) {
        return s1[x]+s2[x>>6]+s3[x>>12];
    }
} T;
```

1.3 Top Tree

FILE data-structures/toptree.cpp **LINES** 60

USE Magic tree that can fulfill every wish. **COST** Tree height O(log n)

AUTHOR ppip

```cpp
vector<pair<int,int>> e[N+5];
int sz[N+5],fa[N+5],son[N+5];
void TOPTREE_INIT(int u) {
    sz[u]=1;
    for (auto [v,_]:e[u])
        if (v!=fa[u]) {
            TOPTREE_INIT(v);
            sz[u]+=sz[v];
            if (sz[v]>sz[son[u]]) son[u]=v;
        }
    if (son[u]) {
        int si{exchange(sz[son[u]],0)};
        sort(e[u].begin(),e[u].end(),[](auto x,auto y){return sz[x.first]<sz[y.first];});
        sz[son[u]]=si;
    }
}
struct cluster {
    //....
} t[N*2+5];
int cnode;
using P=pair<int,int>;
using iter=vector<P>::iterator;
int merge(iter L,iter R,char tp) {
    if (L+1==R) {
        int z{L->first};
        if (z<=n) t[z].u=fa[z],t[z].v=z;
        return z;
    }
    iter p{L+1};
    int sum{accumulate(p,R,-L->second,[](int x,P y){return x+y.second;})};
    while (p+1!=R&&sum>p->second) {
        sum-=p->second*2;
        ++p;
    }
    int u{++cnode};
    t[u].t=tp;
    t[u].l=merge(L,p,tp);
    t[u].r=merge(p,R,tp);
    t[t[u].l].f=t[t[u].r].f=u;
    t[u].u=t[t[u].l].u;
    t[u].v=(tp=='C'?t[t[u].r].v:t[t[u].l].v);
    return u;
}
int build(int u) {
    vector<P> v1;v1.emplace_back(u,1);
    while (son[u]) {
        vector<P> v2;v2.emplace_back(son[u],1);
        for (auto [v,_]:e[u])
            if (v!=fa[u]&&v!=son[u]) v2.emplace_back(build(v),sz[v]);
        v1.emplace_back(merge(v2.begin(),v2.end(),'R'),sz[u]-sz[son[u]]);
        u=son[u];
    }
    return merge(v1.begin(),v1.end(),'C');
}
int main() {
    cnode=n;
    TOPTREE_INIT(1);
    int rt{build(1)};
    return 0;
}
```

2 Geometry

2.1 Geometry basic

FILE Geometry/basic.cpp **LINES** 68

USE 凸包，检查一个点是否在凸包内，闵可夫斯基和

AUTHOR yzj

```cpp
// @complexity
namespace Geometry {
    const db eps = 1e-5;
    struct Vector {
        ll x,y;
        Vector operator+(const Vector &A) { return Vector{x+A.x,y+A.y}; }
        Vector operator-(const Vector &A) { return Vector{x-A.x,y-A.y}; }
        Vector operator*(const Vector &A) { return Vector{x*A.x,y*A.y}; }
        bool operator== (const Vector &A) { return x==A.x && y==A.y; }
        inline ll square() { return x*x+y*y; }
        inline db dis() { return sqrt((db)(x*x+y*y)); }
        inline int section() {
            if(x>0 && y>=0) return 1;
            else if(x<=0 && y>0) return 2;
            else if(x<0 && y<=0) return 3;
            else return 4;
        }
        inline void in() { x = read(); y = read(); }
    };
    const Vector zero = {0,0};
    vc<Vector> as;

    inline ll crs(Vector A,Vector B) { return A.x*B.y-A.y*B.x; }
    inline bool right(Vector A,Vector B) { return crs(A,B)<0; }
    inline bool left (Vector A,Vector B) { return crs(A,B)>0; }
    inline bool cmp1(Vector A,Vector B) { return A.y==B.y?A.x<B.x:A.y<B.y; }
    inline bool cmpangle(Vector A,Vector B) { return A.section()==B.section()?
left(A,B):A.section()<B.section(); }
    inline bool inseg(Vector A,Vector B,Vector C) {
        return fabs((C-A).dis()+(C-B).dis()-(B-A).dis())<eps;
    }

    inline vc<Vector> ConvexHull(vc<Vector> a) {
        sort(a.begin(),a.end(),cmp1);
        vc<Vector> b, c, o; int l=0;
        for(auto u:a) { while(b.size()>1 && right(b[l-1]-b[l-2],u-b[l-1])) b.pop_back(), --l; b.pb(u); +
+l; }
        l=0; reverse(a.begin(),a.end());
        for(auto u:a) { while(c.size()>1 && right(c[l-1]-c[l-2],u-c[l-1])) c.pop_back(), --l; c.pb(u); ++l;}
        o=b; for(int i=1;i<l-1;i++) o.pb(c[i]);
        return o;
    }

    inline vc<Vector> Minkowski(vc<Vector> A,vc<Vector> B) {
        if(!A.size()) return A; if(!B.size()) return B;
        vc<Vector> C; Vector s = A[0]+B[0];
        for(int i=0;i<A.size();i++) C.pb(A[(i+1)%A.size()]-A[i]);
        for(int i=0;i<B.size();i++) C.pb(B[(i+1)%B.size()]-B[i]);
        sort(C.begin(),C.end(),cmpangle);
        vc<Vector> res; Vector sum = s; res.pb(s);
        for(auto u:C) sum = sum+u, res.pb(sum);
        return res.pop_back(), res;
    }

    inline bool in(Vector x) { // ask whether x is in ConvexHull as
        if(as.size()==0) return 0;
        if(as.size()==1) return as[0]==x;
        if(as.size()==2)  return !crs(as[l-1]-as[0],x-as[0]) && inseg(as[l-1],as[0],x);
        int l = as.size();
        if(left(as[l-1]-as[0],x-as[0])) return 0;
        int le = 1, ri = l-2, mid, cnt=0;
        while(le <= ri) {
            ++cnt;
            mid = le+ri>>1;
            if(!right(as[mid]-as[0],x-as[0])) le = mid+1;
            else ri = mid-1;
        }
        return !right(as[ri+1]-as[ri],x-as[ri]);
    }
}
```

3 Graph / Flow

3.1 最大流

FILE graph/flow/maxflow(yzj).cpp **LINES** 27

USE 网络最大流 **COST** O(nm^2), 二分图中 O (m \sqrt n)

AUTHOR yzj

```cpp
namespace flow {
    int dep[N], S, T;
    ll maxflow;
    int hd[N], cur[N], to[N << 1], ne[N << 1], e[N << 1], tc = 1;
    inline void add(int x, int y, int z) {
        to[++tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z;
        to[++tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0;
    }
    inline int bfs() {
        rep(i, 0, T) dep[i] = -1; dep[S] = 0; queue<int> Q; Q.push(S);
        while(!Q.empty()) {
            int x = Q.front(); Q.pop(); cur[x] = hd[x];
            if(x == T) return 1;
            for(int i = hd[x], y; i; i = ne[i])
                if(e[i] && dep[y = to[i]] == -1) dep[y] = dep[x] + 1, Q.push(y);
        }
        return 0;
    }
    inline int dfs(int x, int fl) {
        if(x == T) return maxflow += fl, fl;
        for(int &i = cur[x], y, o; i; i = ne[i])
            if(e[i] && dep[y = to[i]] == dep[x] + 1 && (o = dfs(y, min(fl, e[i])))
                return e[i] -= o, e[i ^ 1] += o, o;
        return 0;
    }
    inline void dinic() { while(bfs()) { while(dfs(S, inf)); } }
}
```

3.2 最大流

FILE graph/flow/maxflow.cpp **LINES** 60

USE Dinic, 要求汇点编号为最大值 **COST** 相信的心就是魔法

AUTHOR ppip

```cpp
struct edge {
    int u,v,w,nxt;
    edge(int _,int __,int ___,int ____):
        u{_},v{__},w{___},nxt{____} {}
    edge() {}
};
vector<edge> e(2);
int hd[N+5];
void _add(int u,int v,int w) {
    e.emplace_back(u,v,w,hd[u]);
    hd[u]=e.size()-1;
}
void add(int u,int v,int w) {
    _add(u,v,w);
    _add(v,u,0);
}
int q[N+5];
int dep[N+5],cur[N+5];
bool bfs(int s,int t) {
    dep[s]=1;
    int l{0},r{0};
    q[r++]=s;
    while (r-l) {
        int u{q[l++]};
        for (int x{hd[u]};x;x=e[x].nxt)
            if (!dep[e[x].v]&&e[x].w) {
                dep[e[x].v]=dep[u]+1;
                q[r++]=e[x].v;
            }
    }
    return dep[t];
}
long long dfs(int s,int t,long long flow) {
    if (s==t||!flow) return flow;
    long long ans{0};
    for (int &x{cur[s]},d;x;x=e[x].nxt) {
        if (dep[e[x].v]==dep[s]+1&&e[x].w&&(d=dfs(e[x].v,t,min(flow,(long long)e[x].w)))) {
```

```
            flow-=d;
            ans+=d;
            e[x].w-=d;
            e[x^1].w+=d;
            if (!flow) return ans;
        }
    }
    return ans;
}
long long calc(int s,int t) {
    fill_n(dep+1,t,0);
    long long flow{0};
    while (bfs(s,t)) {
        copy(hd+1,hd+t+1,cur+1);
        flow+=dfs(s,t,LLONG_MAX);
        fill_n(dep+1,t,0);
    }
    return flow;
}
void clear(int t) {
    e.resize(2);
    fill_n(hd+1,t,0);
}
```

3.3 最小费用最大流

FILE graph/flow/mcmf.cpp **LINES** 38

COST O(nmF)

AUTHOR yzj

```
// @brief
namespace MCMF {
        const int MAXN = 1e6 + 5, inf = 1e18;
        int s, t, mxfl, mico;
        int inq[MAXN], d[MAXN], pr[MAXN];
        int hd[MAXN], cur[MAXN], to[MAXN << 1], ne[MAXN << 1], e[MAXN << 1], c[MAXN << 1], tc = 1;

        inline void add (int x, int y, int z, int w) {
                // cerr << x << " " << y << " " << z << " " << w << endl;
                to[++ tc] = y; ne[tc] = hd[x]; hd[x] = tc; e[tc] = z; c[tc] = w;
                to[++ tc] = x; ne[tc] = hd[y]; hd[y] = tc; e[tc] = 0; c[tc] = - w;
        }

        inline int spfa () {
                rep (i, 0, t) inq[i] = pr[i] = 0, d[i] = inf;
                queue <int> Q; Q.push (s); d[s] = 0; inq[s] = 1;
                while (!Q.empty ()) {
                        int x = Q.front (); Q.pop (); inq[x] = 0; cur[x] = hd[x];
                        for (int i = hd[x], y; i; i = ne[i])
                                if (e[i] && d[y = to[i]] > d[x] + c[i]) {
                                        d[y] = d[x] + c[i]; pr[y] = i;
                                        if (!inq[y]) Q.push (y), inq[y] = 1;
                                }
                }
                return d[t] != inf;
        }

        inline pii mcmf() {
        pii res = mkp(0, 0);
        while(spfa ()) {
            int c = inf;
            for(int x = t; x != s; x = to[pr[x] ^ 1]) c = min(c, e[pr[x]]);
            for(int x = t; x != s; x = to[pr[x] ^ 1]) e[pr[x]] -= c, e[pr[x] ^ 1] += c;
            res.first += c; res.second += d[t] * c;
        }
        return res;
    }
}
```

4 Math

4.1 Binary GCD

FILE math/binary_gcd.cpp **LINES** 6

USE 超快 gcd 算法 **COST** O(log n)，可以当作 O(1)

AUTHOR ppip

```cpp
#define ctz __builtin_ctz
inline int gcd(int a,int b) {
    int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
    while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
    return b<<z;
}
```

4.2 det

FILE math/det.cpp **LINES** 17

USE 行列式求值 **COST** O(n^3)

AUTHOR yzj

```cpp
inline mint det (vc <vc <mint> > a, int n) {
    int rev = 0;
    rep (i, 1, n) {
        rep (j, i + 1, n) {
            while (a[i][i].x) {
                int d = a[j][i].x / a[i][i].x;
                rep (k, i, n) a[j][k] -= a[i][k] * d;
                swap (a[i], a[j]); rev ^= 1;
            }
            swap (a[i], a[j]); rev ^= 1;
        }
        if (!a[i][i].x) return 0;
    }
    mint res = 1;
    rep (i, 1, n) res *= a[i][i];
    return rev ? MOD - res : res;
}
```

5 Math / Number Theory

5.1 Euclid

FILE math/number_theory/euclid.cpp **LINES** 44

COST O(\log max (a, c))

AUTHOR yzj

```cpp
// @brief
namespace Euclid {
    const int mod = 998244353, inv2 = 499122177, inv6 = 166374059;
    struct Data { int f, g, h; };

    inline Data calc(int n, int a, int b, int c) {
        if(a >= c || b >= c) {
            int A = a / c, B = b / c;
            Data t = calc(n, a % c, b % c, c);
            int N = n % mod, s0 = (N + 1) % mod;
            int s1 = N * s0 % mod * inv2 % mod;
            int s2 = N * s0 % mod * (2 * N + 1) % mod * inv6 % mod;
            A %= mod; B %= mod;

            int f = (t.f + A * s1 + B * s0) % mod;
            int g = (t.g + A * A % mod * s2 + B * B % mod * s0
                + 2 * A % mod * t.h + 2 * B % mod * t.f
                + 2 * A % mod * B % mod * s1) % mod;
            int h = (t.h + A * s2 + B * s1) % mod;
            return {f, g, h};
        }

        int m = (a * n + b) / c;
        if(!m) return {0, 0, 0};

        Data t = calc(m - 1, c, c - b - 1, a);
        int N = n % mod, M = m % mod;
        int s1 = N * ((N + 1) % mod) % mod * inv2 % mod;

        int f = (N * M % mod - t.f) % mod;
        int g = (N * M % mod * M % mod - 2 * t.h - t.f) % mod;
        int h = (M * s1 % mod - (t.g + t.f) % mod * inv2) % mod;
```

```
        if(f < 0) f += mod;
        if(g < 0) g += mod;
        if(h < 0) h += mod;
        return {f, g, h};
    }
}

auto [f, g, h] = Euclid::calc(n, a, b, c);

f // sum floor((ai+b)/c)
g // sum floor((ai+b)/c)^2
h // sum i*floor((ai+b)/c)
```

5.2 exgcd

FILE math/number_theory/exgcd.cpp **LINES** 12

COST O(\log n)

AUTHOR yzj

```
// @brief
inline int exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, a;
    int d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
inline int inv(int a, int mod) {
    int x, y;
    exgcd(a, mod, x, y);
    return (x % mod + mod) % mod;
}
```

5.3 min25

FILE math/number_theory/min25.cpp **LINES** 49

COST O(n^{3/4} / \log n)

AUTHOR yzj

```
// @brief
namespace min25 {
    int pri[N], cov[N], sq, m;
    int id1[N], id2[N], val[N], idx;
    mint f[N], g[N], h[N];
    map <pii, mint> dps;

    inline int id (int x) { return x <= sq ? id1[x] : id2[n / x]; }

    inline void init (int n) {
        sq = sqrt (n); m = idx = 0;

        rep (i, 2, sq) {
            if (!cov[i]) pri[++ m] = i, h[m] = h[m - 1] + i;
            for (int j = 1; j <= m && i * pri[j] <= sq; j ++) {
                cov[i * pri[j]] = 1;
                if (i % pri[j] == 0) break;
            }
        }

        for (int l = 1, r, v; l <= n; l = r + 1) {
            val[++ idx] = v = n / l; r = n / v;
            (v <= sq ? id1[v] : id2[n / v]) = idx;
            f[idx] = (mint) v * (v + 1) * inv2 - 1; g[idx] = v - 1;
        }

        rep (j, 1, m) for (int i = 1; i <= idx && pri[j] * pri[j] <= val[i]; i ++) {
            int k = id (val[i] / pri[j]);
            f[i] -= pri[j] * (f[k] - h[j - 1]);
            g[i] -= g[k] - j + 1;
        }
    }

    int TIME = 0;
    inline mint S (int n, int i) {
        if (n <= 1 || pri[i] >= n) return 0;

        int u = id (n);
        mint res = f[u] - g[u] - h[i] + i;
```

```
        if (i == 0) res += 2;

        for (int j = i + 1; j <= m && pri[j] * pri[j] <= n; j ++) {
            for (int c = 1, pw = pri[j]; pw <= n; pw *= pri[j], c ++)
                res += (pri[j] ^ c) * (S (n / pw, j) + (c > 1));
        }

        return res;
    }
}
```

6 Misc

6.1 Default(Yzj)

FILE misc/default(yzj).cpp **LINES** 24

AUTHOR 喵喵喵幼儿园

```
#include<bits/stdc++.h>

#define int long long

#define vc vector
#define pb emplace_back
#define pii pair<int, int>
#define mkp make_pair
#define rep(i, a, b) for(int i = (a); i <= (b); ++i)
#define lep(i, a, b) for(int i = (a); i >= (b); --i)

using namespace std;

mt19937 gen(chrono::system_clock::now().time_since_epoch().count());

inline int read() {
        int x = 0, w = 0; char ch = getchar(); while(!isdigit(ch)) w |= (ch == '-'), ch = getchar();
        while(isdigit(ch)) x = x * 10 + (ch ^ 48), ch = getchar(); return w ? -x : x;
}

signed main() {

        return 0;
}
```

6.2 快速模乘

FILE misc/fastmod.cpp **LINES** 16

USE 要求 1<=P<2^31 且 a<P；同一 z 多次使用时先 trans，再用 mul_t **COST** O(1)

AUTHOR ppip

```
using u32=unsigned;
using u64=unsigned long long;
using u128=unsigned __int128;
const u32 P=998244353;
inline u64 trans(u64 x) {
    constexpr u64 A=-(u64)P/P+1;
    constexpr u64 q=((u128(-(u64)P%P)<<64)+P-1)/P;
    return x*A+u64((u128)x*q>>64)+1;
}
// t 必须是 trans(z) 的返回值；复用同一 z 时只需计算一次 t
inline u32 mul_t(u32 a,u64 t) {
    return a*t*(u128)P>>64;
}
inline u32 mul(u32 a,u64 z) {
    return mul_t(a,trans(z));
}
```

7 Poly

7.1 多项式求逆

FILE poly/finv.cpp **LINES** 97

USE ntt,多项式求逆 **COST** O(nlogn)

AUTHOR ysy

```
#include<iostream>
#include<cstdio>
```

```cpp
#include<cstring>
#define int long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=4000010,mod=998244353,g=3;
int ksm(int p,int k){
    int x=1;
    while(k){
        if(k&1)x=x*p%mod;
        p=p*p%mod;
        k>>=1;
    }
    return x;
}
int r[N];
void init(int lg){
    for(int i=0;i<(1<<lg);i++){
        r[i]=(r[i>>1]>>1)|((i&1)<<(lg-1));
    }
}
void ntt(int *a,int n,bool f=0){
    for(int i=0;i<n;i++){
        if(i<r[i])swap(a[i],a[r[i]]);
    }
    for(int ln=2;ln<=n;ln<<=1){
        int x=ksm(g,(mod-1)/ln),mid=(ln>>1);
        if(f)x=ksm(x,mod-2);
        for(int i=0;i<n;i+=ln){
            int y=1;
            for(int j=0;j<mid;j++){
                int l=a[i+j],r=y*a[i+j+mid]%mod;
                a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
                y=y*x%mod;
            }
        }
    }
    if(f){
        int iv=ksm(n,mod-2);
        for(int i=0;i<n;i++){
            a[i]=a[i]*iv%mod;
        }
    }
}
void mul(int *a,int *b,int n){
    static int c[N],d[N];
    memset(c,0,sizeof(c));
    memset(d,0,sizeof(d));
    int m=(n<<1);
    for(int i=0;i<n;i++){
        c[i]=a[i],d[i]=b[i];
    }
    ntt(c,m,0);ntt(d,m,0);
    for(int i=0;i<m;i++){
        a[i]=c[i]*d[i]%mod;
    }
    ntt(a,m,1);
}
void qinv(int n,int *g){
    int m=1,lg=0;
    static int f[N],h[N];
    memcpy(f,g,sizeof(f));
    memset(g,0,sizeof(g));
    g[0]=ksm(f[0],mod-2);
    while(m<=n){
        m<<=1,lg++;
        init(lg+1);
        memcpy(h,g,sizeof(h));
        memset(g,0,sizeof(g));
        for(int i=0;i<m;i++){
            g[i]=h[i]*2%mod;
```

```
        }
        mul(h,h,m);
        mul(h,f,m);
        for(int i=0;i<m;i++){
            g[i]=(g[i]-h[i]+mod)%mod;
        }
    }
}
int n,a[N],b[N];
signed main(){
    n=read()-1;
    for(int i=0;i<=n;i++){
        a[i]=read();
    }
    qinv(n,a);
    for(int i=0;i<=n;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

7.2 计数常用代码

FILE poly/ntt(yzj).cpp **LINES** 74

USE mint 类和 poly 类 **COST** O(n log n)

AUTHOR yzj

```
const int MOD = 998244353;

// CALC
inline void inc (int &x, int y) { x += y - MOD; x += (x >> 31) & MOD; }
struct mint {
        int x;
        mint (int ix = 0) { x = ix % MOD; if(x < 0) x += MOD; }
        mint operator += (mint a) { inc (x, a.x); return a; }
        mint operator -= (mint a) { inc (x, MOD - a.x); return a; }
        mint operator *= (mint a) { x = 1ll * x * a.x % MOD; return a; }
        mint friend operator + (mint a, mint b) { a += b; return a; }
        mint friend operator - (mint a, mint b) { a -= b; return a; }
        mint friend operator * (mint a, mint b) { a *= b; return a; }
} fc[N], ifc[N];
inline mint qpow (mint a, int b) { mint res = 1; for(; b; b >>= 1, a *= a) if(b & 1) res *= a; return res; }
inline mint qinv (mint a) { return qpow(a.x, MOD - 2); }
inline mint binom (int n, int m) { return n < m ? 0 : fc[n] * ifc[m] * ifc[n - m]; }
//

namespace Poly {
        const int MAXN = 1 << 22;
        int t, bit, rev[MAXN];

        struct poly : vc <mint> {
                poly (int n = 0) { resize (n); for (auto &u : (*this)) u = 0; }
                inline void show () {
                        for (auto u : (*this)) cerr << u.x << " ";
                        cerr << endl;
                }
        } ;

        inline void Resize (poly &a, int n) {
                while (a.size () < n) a.pb (0);
                while (a.size () > n) a.pop_back ();
        }
        inline void init (int n) {
                t = 2, bit = 1; while (t < n) t <<= 1, ++ bit;
                rep (i, 0, t - 1) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (bit - 1));
        }
        inline void dft (poly &a) {
                rep (i, 0, t - 1) if (i < rev[i]) swap (a[i], a[rev[i]]);
                mint o = 1, w = qpow (3, (MOD - 1) >> 1), x, y;
                for (int m = 1; m < t; m <<= 1, w = qpow (3, (MOD - 1) / (m << 1)))
                        for (int i = 0; i < t; i += (m << 1), o = 1) for (int j = 0; j < m; j ++, o = o * w)
                                x = a[i + j], y = o * a[i + j + m], a[i + j] = x + y, a[i + j + m] = x - y;
        }
        inline void idft (poly &a) {
                dft (a); auto inv = qinv (t);
                reverse (a.begin () + 1, a.end ());
```

```
                for (auto &u : a) u *= inv;
        }
        poly operator * (poly a, poly b) {
                int h; init (h = a.size () + b.size () - 1);
                poly f (t); Resize (a, t); Resize (b, t);
                dft (a); dft (b);
                rep (i, 0, t - 1) f[i] = a[i] * b[i];
                idft (f); Resize (f, h);
                return f;
        }
}

int n, c[N], g[N];

inline Poly :: poly solve (int l, int r) {
        if (l == r) {
                Poly :: poly f (c[l] + 1);
                rep (i, 0, min (c[l],  g[l]))
                        if (i & 1) f[c[l] - i] = MOD - binom (c[l], i) * binom (g[l], i) * fc[i];
                        else f[c[l] - i] = binom (c[l], i) * binom (g[l], i) * fc[i];
                return f;
        }
        int mid = ((l + r) >> 1);
        return solve (l, mid) * solve (mid + 1, r);
}
```

7.3 多项式乘法

FILE poly/ntt.cpp **LINES** 73

USE ntt,未经任何优化 **COST** O(nlogn)

AUTHOR ysy

```
#include<iostream>
#include<cstdio>
#define int long long
using namespace std;
int read(){
    int x=0,f=1;char ch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
    return x*f;
}
const int N=4000040,mod=998244353,g=3;
int n,m,a[N],b[N],r[N];
int ksm(int p,int k){
    int x=1;
    while(k){
        if(k&1)x=x*p%mod;
        p=p*p%mod;
        k>>=1;
    }
    return x;
}
void init(int lg){
    for(int i=1;i<(1<<lg);i++){
        r[i]=((r[i>>1]>>1)|((i&1)<<(lg-1)));
    }
}
void ntt(int *a,int lg,int fl=0){
    int n=(1<<lg);
    for(int i=0;i<n;i++){
        if(i<r[i])swap(a[i],a[r[i]]);
    }
    for(int ln=2;ln<=n;ln<<=1){
        int mid=(ln>>1),x=ksm(g,(mod-1)/ln);
        if(fl)x=ksm(x,mod-2);
        for(int i=0;i<n;i+=ln){
            int y=1;
            for(int j=0;j<mid;j++){
                int l=a[i+j],r=a[i+j+mid]*y%mod;
                a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
                y=y*x%mod;
            }
        }
    }
    if(fl){
```

```
        int iv=ksm(n,mod-2);
        for(int i=0;i<n;i++){
            a[i]=a[i]*iv%mod;
        }
    }
}
void mul(){
    int lg=__lg(n+m)+1;
    init(lg);
    ntt(a,lg);ntt(b,lg);
    for(int i=0;i<(1<<lg);i++){
        a[i]=a[i]*b[i]%mod;
    }
    ntt(a,lg,1);
}
signed main(){
    n=read(),m=read();
    for(int i=0;i<=n;i++){
        a[i]=read();
    }
    for(int i=0;i<=m;i++){
        b[i]=read();
    }
    mul();
    for(int i=0;i<=n+m;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

8 String

8.1 manacher

FILE string/manacher.cpp **LINES** 13

USE 线性求回文子串数量 **COST** O (n)

AUTHOR yzj

```
namespace Manacher {
    int r[N], L, R, res;
    inline int manacher(char *s, int n) {
        res = L = R = 0;
        rep(i, 1, n) {
            r[i] = i <= R ? min(r[R + L - i], R - i) : 0;
            while(i + r[i] + 1 <= n && i - r[i] - 1 > 0 && s[i + r[i] + 1] == s[i - r[i] - 1]) ++r[i];
            if(i + r[i] > R) L = i - r[i], R = i + r[i];
            res = max(res, r[i] / 2 * 2 + (i % 2 == 0));
        }
        return res;
    }
}
```

8.2 PAM

FILE string/pam.cpp **LINES** 27

USE 回文树 **COST** O(n)

AUTHOR yzj

```
namespace PAM {
    int ch[N][26], fail[N], len[N], cnt[N], last, tot;
    inline int getfail(int p, int x) {
        while(str[x - len[p] - 1] != str[x]) p = fail[p];
        return p;
    }
    inline void insert(int x) {
        int c = str[x] - 'a', p = getfail(last, x);
        if(!ch[p][c]) {
            len[++tot] = len[p] + 2;
            if(len[tot] == 1) fail[tot] = 0;
            else {
                int q = getfail(fail[p], x);
                fail[tot] = ch[q][c];
            }
            ch[p][c] = tot;
        }
        last = ch[p][c]; cnt[last]++;
```

```
    }
    inline void build() {
        len[0] = 0; fail[0] = 1;
        len[1] = -1; fail[1] = 1;
        last = 0; tot = 1;
        for(int i = 1; i <= n; i++) insert(i);
        for(int i = tot; i > 1; i--) cnt[fail[i]] += cnt[i];
    }
}
```

8.3 SA

FILE string/sa(oscar).cpp **LINES** 41

USE 后缀排序并求出 height 数组 **COST** O(n \log n)

AUTHOR yzj

```
namespace SA {
    int sa[N], rk[N], tmp[N], id[N], buc[N], height[N], idx;
    inline void suffix_sort() {
        for(int i = 0; i < 256; i++) buc[i] = 0;
        for(int i = 1; i <= n; i++) buc[(unsigned char)str[i]]++;
        for(int i = 1; i < 256; i++) buc[i] += buc[i - 1];
        for(int i = n; i; i--) sa[buc[(unsigned char)str[i]]--] = i;
        rk[sa[1]] = idx = 1;
        for(int i = 2; i <= n; i++)
            rk[sa[i]] = str[sa[i]] == str[sa[i - 1]] ? idx : ++idx;

        for(int w = 1; idx < n; w <<= 1) {
            int p = 0;
            for(int i = n - w + 1; i <= n; i++) id[++p] = i;
            for(int i = 1; i <= n; i++) if(sa[i] > w) id[++p] = sa[i] - w;

            for(int i = 1; i <= idx; i++) buc[i] = 0;
            for(int i = 1; i <= n; i++) buc[rk[id[i]]]++;
            for(int i = 1; i <= idx; i++) buc[i] += buc[i - 1];
            for(int i = n; i; i--) sa[buc[rk[id[i]]]--] = id[i];

            for(int i = 1; i <= n; i++) tmp[i] = rk[i];
            rk[sa[1]] = idx = 1;
            for(int i = 2; i <= n; i++) {
                int x = sa[i], y = sa[i - 1];
                rk[x] = tmp[x] == tmp[y] &&
                    (x + w > n ? 0 : tmp[x + w]) == (y + w > n ? 0 : tmp[y + w])
                    ? idx : ++idx;
            }
        }

        for(int i = 1, k = 0; i <= n; i++) {
            if(rk[i] == 1) { k = 0; continue; }
            if(k) k--;
            int j = sa[rk[i] - 1];
            while(i + k <= n && j + k <= n && str[i + k] == str[j + k]) k++;
            height[rk[i]] = k;
        }
        height[1] = 0;
    }
}
```

8.4 后缀数组

FILE string/sa.cpp **LINES** 34

USE 倍增算法求 sa，rk，height, 字符必须不超过 '~' **COST** O(n log n)

AUTHOR ppip

```
char s[N+5];
int id[N*2+5],sa[N+5],buc[N+5],h[N+5],rk[N+5];
int main() {
    // read s
    int n(strlen(s+1));
    for (int i{1};i<=n;++i) ++buc[s[i]];
    partial_sum(buc+1,buc+1+'~',buc+1);
    for (int i{n};i>=1;--i) sa[buc[s[i]]--]=i;
    int m{0};
    for (int i{1};i<=n;++i) rk[sa[i]]=m+=(s[sa[i]]!=s[sa[i-1]]);
    fill_n(buc+1,'~',0);
    for (int w{1};m<n;w<<=1) {
        int cnt{0};
```

```
        for (int i{0};i<w;++i) id[++cnt]=n-i;
        for (int i{1};i<=n;++i) {
            if (sa[i]>w) id[++cnt]=sa[i]-w;
            ++buc[rk[i]];
        }
        partial_sum(buc+1,buc+1+m,buc+1);
        for (int i{n};i>=1;--i) sa[buc[rk[id[i]]]--]=id[i];
        fill_n(buc+1,m,0);
        copy_n(rk+1,n,id+1);
        m=0;
        for (int i{1};i<=n;++i)
            rk[sa[i]]=m+=(id[sa[i]]!=id[sa[i-1]]||id[sa[i]+w]!=id[sa[i-1]+w]);
    }
    // for (int i{1};i<=n;++i) cout<<sa[i]<<" ";
    for (int i{1},k{0};i<=n;++i) {
        k-=!!k;
        while (s[i+k]==s[sa[rk[i]-1]+k]) ++k;
        h[rk[i]]=k;
    }
    return 0;
}
```

8.5 后缀树

FILE string/sft.cpp **LINES** 39

USE Ukkonen 在线构造隐式后缀树 **COST** build O(n)

AUTHOR ppip

```
constexpr int N(1e6),inf(1e7),RNG{26+26+10};
struct Suft {
    int ch[N*2+5][RNG];
    int st[N*2+5],len[N*2+5],link[N*2+5];
    char s[N+5];
    int now{1},rem{0},n{0},tot{1};
    Suft() {len[0]=inf;}
    int new_node(int l,int le) {
        ++tot;st[tot]=l;len[tot]=le;
        return tot;
    }
    void extend(int x) {
        s[++n]=x;++rem;
        for (int lst{1};rem;) {
            while (rem>len[ch[now][s[n-rem+1]]])
                rem-=len[now=ch[now][s[n-rem+1]]];
            int &v{ch[now][s[n-rem+1]]},c{s[st[v]+rem-1]};
            if (!v||x==c) {
                link[lst]=now;lst=now;
                if (!v) v=new_node(n,inf);
                else break;
            } else {
                int u{new_node(st[v],rem-1)};
                ch[u][c]=v;ch[u][x]=new_node(n,inf);
                st[v]+=rem-1;len[v]-=rem-1;
                link[lst]=v=u;lst=u;
            }
            if (now==1) --rem;
            else now=link[now];
        }
    }
    void search(int u,int dep=0) {
        if (st[u]+len[u]-1>=n&&st[u]-dep!=n) cout<<st[u]-dep<<" ";
        else dep+=len[u];
        for (int i{0};i<RNG;++i)
            if (ch[u][i]) search(ch[u][i],dep);
    }

} T;
```

警钟长鸣

读题与建模

- ! 多模拟样例

思考误区

- ! 发现新性质后，回头检查旧做法能否简化或是否已经不成立
- ! DP 先想清楚有哪些可靠的状态和存储方式，再设计转移

实现与边界

- ! 任意范围修改、查询时，考虑复杂度会不会退化

提交之前

- ! 检查多测清空
- ! 检查 long long 和取模
- ! TLE 时重新检查实际复杂度，尤其注意分治树等结构是否退化